

Does an Edge-of-Stability Mechanism Explain E3M4 versus E2M5?

Literature review and implications (9.520, Fall 2026)

Juntang Wang

Víctor Conchello

October 2, 2026

1 E3M4 versus E2M5

1.1 The two formats

An 8-bit $ExMy$ code has a sign bit s , an exponent field E of x bits and a mantissa M of y bits. Given an exponent bias b , a code with $E > 0$ is normal and represents $(-1)^s 2^{E-b} (1.M)_2$. A code with $E = 0$ is subnormal and fills the gap between zero and the smallest normal value. For the bias and the special values, we use the OCP 8-bit floating-point (OFP8) convention [15]. The bias is $b = 2^{x-1} - 1$. There are no infinities and only the all-ones pattern encodes NaN. The bias only shifts the representable window; its width and spacing are what separate the two formats.

	E3M4	E2M5
Unit roundoff (round-to-nearest), $u = 2^{-(y+1)}$	$2^{-5} \approx 3.1\%$	$2^{-6} \approx 1.6\%$
Powers of two covered by normal values	7	3
Largest / smallest positive value	$30 / 2^{-6}$	$7.75 / 2^{-5}$
Dynamic range, largest/smallest nonzero	$\approx 2^{11}$	$\approx 2^8$

Table 1: The two formats under the OFP8 convention.

1.2 How the literature distinguishes the two formats

Chmiel et al. [3]. *E3M4 works better than E2M5 as the scale format for FP4 training.* This study trains LLMs with FP4 (E2M1) weights, activations and gradients in all three matrix multiplications: forward, backward and update. Each block of FP4 values shares one 8-bit scale, which multiplies the block back to its real magnitude when the values enter a higher-precision computation. E3M4 and E2M5 scales have no hardware support, so they are simulated. Block sizes and scale formats are swept on a 350M Llama-style model with block size 16. The scale format takes seven values. E3M4 and E4M3 reach the lowest loss. E2M5, E5M2 and E8M0 end slightly worse, E6M1 clearly worse, and E1M6 diverges. This challenges the Microscaling (MX) standard [18], which uses a power-of-two E8M0 scale with no mantissa bits. A scale error rescales all 16 elements of its block together, so it produces correlated rather than independent perturbations. The scales are positive, so the sign bit is unused. The recipe uses round-to-nearest in the forward pass and stochastic rounding for neural gradients (backward and update matrix multiplications) and for activations in the update matrix multiplication. The authors also show that once the gradient norm falls below about $\sqrt{3}$ times the quantization noise, a short higher-precision phase is needed. The format ranking itself is purely empirical: the authors report it without explaining it.

Kuzmin et al. [14]. *More exponent bits pay off when tensors have outliers; more mantissa bits pay off otherwise.* This paper derives the expected quantization error of FP8 formats as a function of the tensor distribution: for Gaussian tensors E2M5 is optimal, and the optimum moves to more

exponent bits as outliers grow. Pretrained FP32 transformers have heavy-tailed activations, so their post-training quantization favours more exponent bits. With a bias set per channel, E3M4 beats E2M5 on ViT (77.7 vs 77.3) and on BERT (82.6 vs 80.3); with a single bias for the whole network, E2M5 falls to 51.1 on ViT against 77.0 for E3M4. For CNNs the ranking depends on the bias. Quantization-aware training narrows the gaps between formats to about one accuracy point or less, but does not close them. This is a first hint that a format gap can partly reflect a mismatch between the training and deployment arithmetic.

Role of the format, by tensor. Early FP8 training work already chose the format according to the tensor: precision for weights and activations, range for gradients. Sun et al. [20] used a 1-4-3 format in the forward pass and 1-5-2 in the backward pass. Noune et al. [16] found that the exponent bias must be tuned, with more mantissa bits for weights and activations and more range for gradients. Mickevicius et al. [15] proposed E4M3 and E5M2 with per-tensor scaling, later standardized by OCP as OFP8.

Distribution and outliers. Outlier features that grow with model scale [7] widen the spread of block maxima, which favours range (E3M4). For well-behaved inference tensors, van Baalen et al. [21] find INT8 often preferable to FP8, and E2M5 is the more integer-like of the two formats, with fewer and longer binades. Format effects can also appear late. Fishman et al. [9] trace FP8 divergences in trillion-token runs to outlier amplification in SwiGLU, which short sweeps such as Chmiel et al.’s would miss.

Rounding, accumulation and saturation. Round-to-nearest loses updates smaller than half a grid step, while stochastic rounding is unbiased; with 16-bit fixed point, only stochastic rounding matches FP32 training [10]. Neither is better everywhere: testing each tensor separately, Chmiel et al. [3] find that stochastic rounding lowers the loss for neural gradients but raises it in the forward pass. Saturation is a separate failure mode. Su et al. [19] trace loss spikes in microscaling training to values clamped at the top of the scaled range, and show that at 8 bits E4M3 beats E5M2. In both settings, more mantissa bits help as long as range suffices.

2 The edge of stability

Cohen et al. [4]. *Gradient descent drives sharpness up to the stability limit $2/\eta$ and then keeps it there while the loss still falls.* For a quadratic with curvature λ , a gradient step multiplies the error by $1 - \eta\lambda$, so it converges only if the sharpness $\lambda_{\max}(\nabla^2 L) < 2/\eta$. Cohen et al. show that full-batch gradient descent violates the assumption behind classical analyses. Sharpness first rises (*progressive sharpening*), then settles at or just above $2/\eta$. The loss oscillates along the top Hessian eigenvector q_1 but still decreases over long horizons. The next top eigenvalues follow, rising to $2/\eta$ one after another and staying there. With heavy-ball momentum the threshold becomes $(2 + 2\beta)/\eta$. Under SGD, sharpness does not settle at the threshold.

Damian et al. [6]. *Oscillations at the edge of stability reduce sharpness automatically, so training stays stable.* If the iterate moves a distance x along q_1 , the cubic Taylor term adds about $\frac{x^2}{2} \nabla^3 L[q_1, q_1] = \frac{x^2}{2} \nabla \lambda_{\max}$ to the gradient, so gradient descent moves along $-\nabla \lambda_{\max}$. Oscillations along q_1 therefore reduce sharpness until the iterates are stable again. In aggregate, gradient descent follows projected gradient descent on $\{\theta : \lambda_{\max}(\theta) \leq 2/\eta, \nabla L \cdot q_1 = 0\}$. With several eigenvalues at the edge, training becomes more chaotic. The authors note that the dynamics are highly sensitive to small perturbations and switch to 64-bit gradients once training reaches instability, to avoid

propagating floating-point error. That makes numerical precision a direct concern for any EoS-based explanation.

Cohen et al. [5]. *Adam also trains at the edge of stability, measured on the preconditioned sharpness.* For Adam with preconditioner $P = \text{diag}(\sqrt{\nu}) + \epsilon I$, where ν is the running average of squared gradients, the relevant quantity is the preconditioned sharpness $\lambda_{\max}(P^{-1}H)$. With P held fixed, Adam on a quadratic diverges once it exceeds $2(1 + \beta_1)/(\eta(1 - \beta_1)) = 38/\eta$ for $\beta_1 = 0.9$, the analogue of $2/\eta$. In full-batch experiments, the preconditioned sharpness rises to $38/\eta$ and hovers there while the loss keeps falling, just as sharpness does at $2/\eta$ under gradient descent. AdamW is the standard optimizer for LLM training, Chmiel et al.’s included, so this paper is what makes the edge of stability relevant to modern LLM training.

Earlier evidence and linear stability. Wu et al. [23] showed that gradient descent can only remain at minima with sharpness $\leq 2/\eta$, and SGD faces an additional constraint from gradient-noise non-uniformity. Jastrzębski et al. [12] found that, early in training, SGD reaches a point where its steps become too large for the sharpest direction, which they call the *break-even point*.

Theory of why training continues. Arora et al. [2] prove that, for normalized gradient descent and for gradient descent run on the transformed loss $\sqrt{L - \min L}$ instead of the loss L itself, the iterates enter the edge of stability and follow a sharpness-reducing flow on the manifold of minimizers. Ahn et al. [1] characterize unstable convergence and the conditions that make it possible. Kreisler et al. [13] prove that, in scalar networks, gradient descent monotonically decreases the sharpness of the gradient-flow solution it would reach.

2.1 Has the edge of stability been used to compare number formats?

We found no study that measures sharpness while varying number format. The closest is Su et al. [19]. They model the low-precision gradient as $(1 + \zeta_t)\bar{g}_t$ and derive a heuristic stability condition $|1 - \eta\lambda_{\max}| + \eta\|\zeta_t\|\lambda_{\max} \lesssim 1$. When $\eta\lambda_{\max} > 1$, this shrinks the stable step size to $2/((1 + \|\zeta_t\|)\lambda_{\max})$. They estimate a lower bound on $\|\zeta_t\|$ from paired FP32 and MXFP8 runs and find that the loss diverges once it reaches about 2. They frame this as a smaller EoS region, but never measure sharpness, so they do not check it against Cohen et al. [4]. Wortsman et al. [22], studying int8 training of CLIP models, trace loss spikes to AdamW second-moment estimates that underestimate the squared gradients (mainly in the patch-embedding layer), and note that these spikes come with large activations and gradients that strain low-precision formats. That is a preconditioner-based account, not a format-specific curvature measurement. An EoS explanation of the format gap would require E2M5 to perturb training along q_1 , or to bias P , more than E3M4 does. No study has tested this.

2.2 A complementary mechanism: training–inference mismatch

Qi et al. [17]. *Low precision destabilizes RL fine-tuning through a mismatch between the training and inference engines, and FP16 fixes it.* In RL fine-tuning of LLMs, the rollout engine (e.g. vLLM) and the training engine compute different token probabilities for the same weights, which biases the policy ratio and can collapse training. Algorithmic corrections only partially help, whereas switching both engines from BF16 to FP16 (7 versus 10 mantissa bits) largely removes the problem across algorithms and frameworks. Yao et al. [24] document the same gap and correct it with truncated importance sampling, and He and Thinking Machines Lab [11] show that the mismatch can arise even at fixed precision, from inference kernels that are not batch-invariant.

3 Implications for machine learning theory and practice

1. Stability theory must model the arithmetic, not only the step size. EoS theory [4–6] assumes exact gradients, so its threshold is $2/\eta$ (or $38/\eta$ preconditioned) for every format. A format can only matter through the error it injects, and what matters is the error’s component along q_1 and its bias, not its average size. Su et al.’s multiplicative model lowers the threshold to $2/((1 + \|\zeta\|)\eta)$. With shared block scales the error is correlated within each block, so its projection on q_1 can be much larger than isotropic-noise models predict. This yields a falsifiable prediction. If EoS explains the gap, E2M5 should equilibrate at a measurably lower sharpness (or preconditioned sharpness) than E3M4 at the same η . If both settle at the same threshold, curvature is not the channel, and the gap must come from range loss, mismatch, or a slower-converging but equally stable trajectory. Measuring this requires sharpness computed in high precision (Lanczos with Hessian–vector products in FP32/FP64), since Damian et al. note that the dynamics are sensitive to small perturbations and switch to 64-bit gradients to avoid propagating floating-point error.

2. Explaining the E3M4 advantage would turn format choice from a sweep into a prediction. Today the ranking of scale formats is purely empirical: Chmiel et al. [3] find E3M4 ahead of E2M5 without explaining why, and Kuzmin et al. [14] show that the ranking changes with the bias and the tensor distribution. Our project asks which mechanism produces the gap, and each answer has a different consequence. If the gap comes from range, the ranking should follow from tensor statistics, such as the spread of block maxima that a scale must encode, and could be predicted before training. If it comes from the edge of stability, the ranking also depends on the learning rate, and should shrink or vanish when $\eta\lambda_{\max}$ stays well below threshold. For theory, either answer replaces a sweep with a principle that can be tested on new formats. For practice, it tells hardware designers which bits to spend: Chmiel et al.’s results already challenge the MX standard’s E8M0 scale, with no mantissa bits, and a mechanism would say whether E3M4 is the right replacement for every model or only for some training regimes.

3. Optimizer-state precision is a stability parameter, not only a memory saving. Under Adam, the edge of stability is set by the preconditioned sharpness $\lambda_{\max}(P^{-1}H)$ [5], so any error in $P = \text{diag}(\sqrt{\nu}) + \epsilon I$ moves the effective threshold. Memory-efficient training stores ν in 8 bits [8], yet ν spans many orders of magnitude across parameters. A format with too little range flushes the small entries of each block to zero, so P falls to ϵI in those coordinates, their steps grow by orders of magnitude, and $\lambda_{\max}(P^{-1}H)$ can cross $38/\eta$ at a learning rate that is otherwise stable. This gives a second, concrete route by which E3M4 could beat E2M5: through the optimizer state rather than the gradients. It is testable by quantizing only ν in each format and tracking $\lambda_{\max}(P^{-1}H)$ against $38/\eta$. In practice, ν should be kept in a range-rich format, or its underflow rate monitored.

4. Practical consequences for low-precision training and RL. Because the stable step size can depend on the format, comparing formats at one shared learning rate confounds format with $\eta\lambda$. Learning rates and warmup should be tuned per format, or at least swept. Following Qi et al. [17], rollout and training engines should share arithmetic, or their discrepancy should be monitored, whenever probabilities are compared across engines. Finally, monitoring sharpness online is affordable with a few Hessian–vector products every few hundred steps, and could serve as an early warning before the late divergences reported by Fishman et al. [9].

References

- [1] K. Ahn, J. Zhang, and S. Sra. Understanding the unstable convergence of gradient descent. In *ICML*, 2022. arXiv:2204.01050.
- [2] S. Arora, Z. Li, and A. Panigrahi. Understanding gradient descent on edge of stability in deep learning. In *ICML*, 2022. arXiv:2205.09745.
- [3] B. Chmiel, M. Fishman, R. Banner, and D. Soudry. FP4 all the way: Fully quantized training of large language models. In *NeurIPS*, 2025. arXiv:2505.19115.
- [4] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. In *ICLR*, 2021. arXiv:2103.00065.
- [5] J. M. Cohen, B. Ghorbani, S. Krishnan, N. Agarwal, S. Medapati, M. Badura, D. Suo, D. Cardoze, Z. Nado, G. E. Dahl, and J. Gilmer. Adaptive gradient methods at the edge of stability. arXiv:2207.14484, 2022.
- [6] A. Damian, E. Nichani, and J. D. Lee. Self-stabilization: The implicit bias of gradient descent at the edge of stability. In *ICLR*, 2023. arXiv:2209.15594.
- [7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *NeurIPS*, 2022. arXiv:2208.07339.
- [8] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer. 8-bit optimizers via block-wise quantization. In *ICLR*, 2022. arXiv:2110.02861.
- [9] M. Fishman, B. Chmiel, R. Banner, and D. Soudry. Scaling FP8 training to trillion-token LLMs. In *ICLR*, 2025. arXiv:2409.12517.
- [10] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan. Deep learning with limited numerical precision. In *ICML*, 2015.
- [11] H. He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. *Thinking Machines Lab: Connectionism*, 2025. <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.
- [12] S. Jastrzębski, M. Szymczak, S. Fort, D. Arpit, J. Tabor, K. Cho, and K. Geras. The break-even point on optimization trajectories of deep neural networks. In *ICLR*, 2020. arXiv:2002.09572.
- [13] I. Kreisler, M. S. Nacson, D. Soudry, and Y. Carmon. Gradient descent monotonically decreases the sharpness of gradient flow solutions in scalar networks and beyond. In *ICML*, 2023. arXiv:2305.13064.
- [14] A. Kuzmin, M. van Baalen, Y. Ren, M. Nagel, J. Peters, and T. Blankevoort. FP8 quantization: The power of the exponent. In *NeurIPS*, 2022. arXiv:2208.09225.
- [15] P. Micikevicius, D. Stosic, N. Burgess, et al. FP8 formats for deep learning. arXiv:2209.05433, 2022.
- [16] B. Noun, P. Jones, D. Justus, D. Masters, and C. Luschi. 8-bit numerical formats for deep neural networks. arXiv:2206.02915, 2022.
- [17] P. Qi, Z. Liu, X. Zhou, T. Pang, C. Du, W. S. Lee, and M. Lin. Defeating the training-inference mismatch via FP16. arXiv:2510.26788, 2025.
- [18] B. Darvish Rouhani, R. Zhao, A. More, et al. Microscaling data formats for deep learning. arXiv:2310.10537, 2023.
- [19] H. Su, M. Kwun, S. Gil, S. Kakade, and N. Anand. Characterization and mitigation of training instabilities in microscaling formats. arXiv:2506.20752, 2025.
- [20] X. Sun, J. Choi, C.-Y. Chen, et al. Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks. In *NeurIPS*, 2019.
- [21] M. van Baalen, A. Kuzmin, et al. FP8 versus INT8 for efficient deep learning inference. arXiv:2303.17951, 2023.
- [22] M. Wortsman, T. Dettmers, L. Zettlemoyer, A. Morcos, A. Farhadi, and L. Schmidt. Stable and low-precision training for large-scale vision-language models. In *NeurIPS*, 2023. arXiv:2304.13013.
- [23] L. Wu, C. Ma, and W. E. How SGD selects the global minima in over-parameterized learning: A dynamical stability perspective. In *NeurIPS*, 2018.
- [24] F. Yao, L. Liu, D. Zhang, C. Dong, J. Shang, and J. Gao. Your efficient RL framework secretly brings you off-policy RL training. Blog post, 2025. <https://fengyao.notion.site/off-policy-rl>.